

C++ 프로그래밍

함수 오버로딩, 생성자 오버로딩, 연산자 오버로딩

함수 시그니처 (Function Signature)

컴파일러는 **함수 시그니처**로 함수를 식별한다. 시그니처는 함수 이름과 매개변수 목록으로 구성된다.

시그니처를 구성하는 것

- 함수 이름
- 매개변수의 개수
- 매개변수의 타입
- 매개변수의 순서

```
void print(int n);  
void print(double x);  
void print(int n, double x);  
void print(double x, int n);
```

위 네 함수는 시그니처가 모두 다르다.

시그니처에 포함되지 않는 것

- 반환형 — 반환형만 다른 함수는 동일한 시그니처

```
int getValue(); // ✗  
double getValue(); // ✗ 시그니처 충돌 - 컴파일 오류
```

- 매개변수 이름 — 이름은 식별에 사용되지 않음

```
void move(int x, int y); // 시그니처:  
void move(int a, int b); // move(int, int) - 동일
```

컴파일러는 호출 시 **인수의 타입과 개수**를 보고 어떤 함수를 호출할지 결정한다.

함수 오버로딩 (Function Overloading)

같은 이름의 함수를 매개변수 구성이 다르게 여러 개 정의하는 것. 컴파일러가 호출 시점에 인수 타입·개수로 어떤 함수를 호출할지 결정한다 (오버로드 해석).

```
#include <stdio>

void print(int n)          { printf("int: %d\n", n); }
void print(double x)       { printf("double: %.2f\n", x); }
void print(const char* s)  { printf("string: %s\n", s); }
void print(int n, int m)   { printf("two ints: %d, %d\n", n, m); }

int main() {
    print(42);              // print(int)          → int: 42
    print(3.14);           // print(double)        → double: 3.14
    print("hello");        // print(const char*)    → string: hello
    print(1, 2);           // print(int, int)       → two ints: 1, 2
}
```

조건

오버로딩 가능?

매개변수 개수 다름



매개변수 타입 다름



매개변수 순서 다름



반환형만 다름

✗ 컴파일 오류

생성자 오버로딩

복사 생성자, 복사 대입 연산자

생성자 오버로딩

같은 이름의 생성자를 매개변수 구성이 다르게 여러 개 정의할 수 있다.

```
class Point2D {
public:
    int x, y;

    Point2D()                : x(0), y(0) {}           // 기본 생성자
    Point2D(int _x, int _y)   : x(_x), y(_y) {}         // 2인수
    Point2D(const Point2D& other) : x(other.x), y(other.y) {} // 복사 생성자
};

int main() {
    Point2D a;           // 기본 생성자 → (0, 0)
    Point2D b(3, 4);     // 2인수      → (3, 4)
    Point2D c = b;       // 복사 생성자 → (3, 4)
    Point2D d(c);        // 복사 생성자 → (3, 4) (동일)
}
```

- 매개변수 개수 또는 타입이 다르면 동일한 이름으로 정의 가능
- 복사 생성자도 오버로딩의 한 형태 — `const T&` 를 받는 생성자

복사 대입 연산자

객체를 다른 객체로부터 복사할 때 자동으로 호출되는 특수 멤버 함수다.

```
class Point2D {
public:
    int x, y;

    Point2D() : x(0), y(0) {} // 기본 생성자
    Point2D(int _x, int _y) : x(_x), y(_y) {} // 2인수
    Point2D(const Point2D& other) : x(other.x), y(other.y) {} // 복사 생성자

    // 복사 대입 연산자 - *this 반환으로 체이닝 지원
    Point2D& operator=(const Point2D& other) {
        x = other.x;
        y = other.y;
        return *this;
    }
};

int main() {
    Point2D a(3, 4);
    Point2D b, c;

    b = a; // 복사 대입 연산자 호출
    c = b = a; // 체이닝: b=a 후 c=b
}
```

Point2D 처럼 동적 메모리가 없으면 컴파일러가 자동 생성한 버전으로 충분하다.

깊은 복사 (Deep Copy)

포인터 멤버가 있는 클래스는 ****기본 복사(얕은 복사)****가 포인터 주소만 복사해 문제를 일으킨다. 깊은 복사가 필요한 경우 **복사 생성자와 복사 대입 연산자** 양쪽 모두에 구현해야 한다.

얕은 복사의 문제

```
class Vector {
public:
    double* data;
    int size;
    Vector(int n) : size(n) {
        data = new double[n]();
    }
    ~Vector() { delete[] data; }
    // 복사 생성자/대입 없음 → 포인터만 복사됨
};

Vector a(3);
a.data[0] = 1.5;
Vector b = a;      // ⚠ data 포인터만 복사
// a.data == b.data (같은 메모리 참조)
// 소멸 시 같은 메모리를 두 번 delete → 크래시!
```

깊은 복사 구현

```
class Vector {
public:
    double* data;
    int size;
    // ...생성자/소멸자 동일...

    // 복사 생성자 - operator=45 에 위임
    // data를 nullptr로 초기화 필수: 미초기화 포인터에 delete[] 하면 UB
    Vector(const Vector& other) : size(0), data(nullptr) {
        *this = other;
    }

    // 복사 대입 연산자 - 깊은 복사 로직은 여기에만
    Vector& operator=(const Vector& other) {
        if (this == &other) return *this; // 자기 대입 방지
        delete[] data;                    // nullptr도 안전한 no-op
        size = other.size;
        data = new double[size];
        memcpy(data, other.data, size * sizeof(double));
        return *this;
    }
};
```

복사 생성자만 구현하고 복사 대입 연산자를 빠뜨리면 `b = a` 시 얕은 복사가 발생한다. 반드시 **둘 다** 구현하라.

static 멤버

static 멤버는 클래스 자체에 속하는 멤버로, 모든 인스턴스가 공유한다.

static 멤버 변수

```
class Counter {
public:
    static int count; // 클래스 안에서는 선언만
    int id; // 인스턴스마다 별도로 존재

    Counter() {
        ++count; // static - 모든 인스턴스가 공유
        id = count; // non-static - 이 인스턴스의 고유 번호
    }
    ~Counter() { --count; }
};

// 클래스 외부에서 정의 필수
int Counter::count = 0;

int main() {
    Counter a, b;
    printf("%d %d\n", a.id, b.id); // 1 2
    printf("%d\n", Counter::count); // 2
}
```

static 멤버 함수

```
class Counter {
public:
    static int count;
    int id;

    Counter() { ++count; id = count; }
    ~Counter() { --count; }

    static int get() {
        return count; // ✅ static 멤버 접근 가능
        // return id; // ❌ this 없음 → non-static 접근 불가
    }
};

int Counter::count = 0;

int main() {
    Counter a, b;
    printf("id: %d %d\n", a.id, b.id); // id: 1 2
    printf("count: %d\n", Counter::get()); // count: 2
}
```

생성자·소멸자는 this 가 있으므로 static·non-static 모두 접근 가능.

static 함수는 this 가 없으므로 static 멤버에만 접근할 수 있다.

const 멤버 함수

함수 선언 뒤 `const` 는 이 함수는 객체를 수정하지 않는다는 계약이다.

```
class Circle {
    double radius;
public:
    Circle(double r) : radius(r) {}

    // const - 읽기 전용, 멤버 수정 불가
    double area() const {
        return 3.14159 * radius * radius;
    }
    double perimeter() const {
        return 2.0 * 3.14159 * radius;
    }

    // 비-const - 멤버 변수 수정 가능
    void set(double r) { radius = r; }
};
```

```
Circle c(5.0);
const Circle cc(3.0);

c.area();          // ✔ 일반 객체 → const 함수 호출 가능
c.set(10.0);       // ✔ 일반 객체 → 비-const 함수 호출 가능

cc.area();         // ✔ const 객체 → const 함수만 호출 가능
// cc.set(1.0);    // ✗ const 객체에 비-const 함수 불가
```

	const 객체	일반 객체
const 함수	✔	✔
비-const 함수	✗	✔

값을 읽기만 하는 멤버 함수에는 항상 `const` 를 붙이는 것이 관례다.

연산자 오버로딩 기초

연산자 (Operator)

이항 연산자 (Binary Operator)

피연산자가 **두 개**인 연산자.

예) $3 + 5$

```
1  a + b  a - b  a * b  a / b
2  a == b a != b a < b  a > b
3  a = b  a += b a[i]
```

단항 연산자 (Unary Operator)

피연산자가 **한 개**인 연산자.

예) -3

```
1  -a  !a  ~a
2  ++a a++ (전위/후위 구분)
3  --a a--
```

연산자 오버로딩 (Operator Overloading)

C++는 내장 연산자를 사용자 정의 타입에 맞게 재정의할 수 있다.

`operator` 키워드에 연산자 기호를 붙여 정의한다.

```
class Complex {
public:
    double re, im;
    Complex(double re, double im = 0.0) : re(re), im(im) {}

    // 이항 연산자
    Complex operator+(const Complex& o) const {
        return Complex(re + o.re, im + o.im);
    }
    // 단항 연산자: 부호 반전 (-c)
    Complex operator-() const {
        return Complex(-re, -im);
    }
};

Complex a(1.0, 2.0);    // 1 + 2i
Complex b(3.0, -1.0);  // 3 - i

Complex c = a + b;      // a.operator+(b)
Complex d = -c;
```

이항 연산자 오버로딩

두 피연산자를 받는 연산자다. **멤버 함수**로 구현할 때 왼쪽 피연산자는 `this`, 오른쪽은 매개변수가 된다.

```
class Complex {
public:
    double re, im;
    Complex(double re, double im = 0.0) : re(re), im(im) {}

    Complex operator+(const Complex& o) const {
        return Complex(re + o.re, im + o.im);
    }
    Complex operator-(const Complex& o) const {
        return Complex(re - o.re, im - o.im);
    }
    Complex operator*(const Complex& o) const {
        return Complex(re * o.re - im * o.im,
                        re * o.im + im * o.re);
    }
    bool operator==(const Complex& o) const {
        return re == o.re && im == o.im;
    }
};
```

```
Complex a(1.0, 2.0); // 1 + 2i
Complex b(3.0, -1.0); // 3 - i

Complex c = a + b; // (4.0 + 1.0i)
Complex d = a * b; // (5.0 + 5.0i)

printf(a == b ? "같은\n" : "다름\n"); // 다름
```

연산자	반환형 권장	비고
<code>operator+</code> , <code>-</code> , <code>*</code>	<code>T</code>	새 객체 반환
<code>operator+=</code> , <code>-=</code>	<code>T&</code>	자신 수정 후 반환
<code>operator==</code> , <code>!=</code>	<code>bool</code>	
<code>operator=</code>	<code>T&</code>	<code>*this</code> 반환

의미가 직관적으로 명확할 때만 오버로딩하라.

단항 연산자 오버로딩

피연산자가 하나인 연산자다. 전위/후위 ++ 는 **더미** int **매개변수**로 구분한다.

```
class Complex {
public:
    double re, im;
    Complex(double re, double im = 0.0) : re(re), im(im) {}

    // 부호 반전 (-c)
    Complex operator-() const {
        return Complex(-re, -im);
    }

    // 전위 증가 (++c) - 증가 후 자신 반환
    Complex& operator++() {
        re += 1.0;
        return *this;
    }

    // 후위 증가 (c++) - 반환 후 증가, int 더미로 구분
    Complex operator++(int) {
        Complex tmp = *this;
        re += 1.0;
        return tmp;
    }
};
```

```
Complex c(3.0, 4.0);    // 3 + 4i

Complex neg = -c;
// neg: -3.0 + -4.0i,  c: 3.0 + 4.0i (불변)

Complex pre = ++c;
// c: 4.0 + 4.0i (먼저 증가)
// pre: 4.0 + 4.0i (증가된 값)

Complex post = c++;
// post: 4.0 + 4.0i (증가 전 값)
// c: 5.0 + 4.0i (나중에 증가)
```

전역 함수로 연산자 오버로딩

피연산자 순서가 좌우 대칭이어야 하거나,

수정 불가능한 클래스(ostream 등)가 왼쪽 피연산자일 때 전역 함수로 구현한다.

```
#include <iostream>

class Complex {
public:
    double re, im;
    Complex(double re, double im = 0.0) : re(re), im(im) {}
};

// 전역 함수 - 두 피연산자 순서 모두 지원 (대칭 연산)
Complex operator*(const Complex& c, double s) {
    return Complex(c.re * s, c.im * s);
}
Complex operator*(double s, const Complex& c) {
    return c * s;    // 위 함수 재사용
}

// ostream은 외부 클래스 → 멤버 함수 추가 불가
// re, im이 public이므로 friend 없이 접근 가능
std::ostream& operator<<(std::ostream& os, const Complex& c) {
    os << c.re << " + " << c.im << "i";
    return os;    // 체이닝 지원: cout << a << b
}

Complex c(3.0, 4.0);

std::cout << c << "\n";    // 3 + 4i
std::cout << c * 2.0 << "\n";    // 6 + 8i
std::cout << 2.0 * c << "\n";    // 6 + 8i
```

friend 선언

friend 로 지정된 함수나 클래스는 private / protected 멤버에 직접 접근할 수 있다.

friend 함수

```
class Box {
    int width;
public:
    Box(int w) : width(w) {}

    // 외부 함수를 friend로 선언
    friend void print_width(const Box& b);
};

// Box의 private 멤버 직접 접근 가능
void print_width(const Box& b) {
    printf("width: %d\n", b.width);
}
```

friend 클래스

```
class Car {
    int speed = 0;
public:
    friend class Mechanic; // Mechanic에 전체 접근 허용
};

class Mechanic {
public:
    void turbo(Car& c) {
        c.speed += 100; // ✅ private 접근 가능
        printf("tuned: %d\n", c.speed);
    }
};
```

friend 는 캡슐화를 약화시키므로 최소화해야 한다. 주로 operator<< 같은 연산자 오버로딩이나 긴밀하게 협력하는 클래스 쌍에서 사용한다.

friend와 연산자 오버로딩

멤버가 `private` 일 때, 전역 연산자 함수가 해당 멤버에 접근하려면 `friend` 선언이 필요하다.

문제: private + 전역 연산자

```
class Complex {
    double re, im;    // private
public:
    Complex(double re, double im = 0.0)
        : re(re), im(im) {}
};

// ✗ re, im이 private → 전역 함수에서 접근 불가
Complex operator+(const Complex& a,
                  const Complex& b) {
    return Complex(a.re + b.re,    // 컴파일 오류
                  a.im + b.im);
}

std::ostream& operator<<(std::ostream& os,
                        const Complex& c) {
    os << c.re << " + " << c.im << "i"; // 컴파일 오류
    return os;
}
```

해결: friend로 접근 허용

```
class Complex {
    double re, im;    // private 유지
public:
    Complex(double re, double im = 0.0)
        : re(re), im(im) {}

    // 전역 연산자에 private 접근 허용
    friend Complex operator+(const Complex& a,
                             const Complex& b);
    friend std::ostream& operator<<(std::ostream& os,
                                     const Complex& c);
};

Complex operator+(const Complex& a, const Complex& b) {
    return Complex(a.re + b.re, a.im + b.im); // ✔️
}

std::ostream& operator<<(std::ostream& os,
                        const Complex& c) {
    os << c.re << " + " << c.im << "i";    // ✔️
    return os;
}
```

(혹은) 클래스 멤버변수를 `public` 으로 공개하던지

(혹은) 클래스 멤버함수로 getter 함수를 구현

연산자 오버로딩 — 멤버 vs 전역

구분	해당 연산자	이유
반드시 멤버 함수	<code>=</code> <code>[]</code> <code>()</code> <code>-></code>	표준이 멤버 함수로 명시
사실상 전역 함수	<code><<</code> <code>>></code>	왼쪽 피연산자가 <code>ostream</code> / <code>istream</code> — 수정 불가한 외부 클래스
멤버 또는 전역 모두 가능	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>==</code> <code>!=</code> <code><</code> <code>></code> <code>+=</code> 등	선택 가능; 대칭 연산이면 전역 권장

전역 함수만 가능한 연산자는 C++에 없다. `<<` / `>>` 는 기술적으로 멤버로도 정의할 수 있지만, 왼쪽 피연산자인 `ostream` 을 수정할 수 없으므로 사실상 전역으로 구현해야 한다.

연산자 오버로딩 심화

operator[] — 인덱스 접근 연산자

배열처럼 `v[i]` 문법으로 원소에 접근할 수 있게 한다. `const` 오버로딩을 함께 제공하는 것이 관례다.

구현

```
class Vector {
    double* data;
    int     size;
public:
    Vector(int n) : size(n) { data = new double[n](); }
    ~Vector() { delete[] data; }

    // 비-const 객체: double& 반환 → 쓰기 가능
    double& operator[](int i) { return data[i]; }

    // const 객체: 값 반환 → 읽기 전용
    double operator[](int i) const { return data[i]; }
};
```

사용

```
Vector v(3);

v[0] = 1.414;           // 쓰기 - double& 반환이므로 대입 가능
v[1] = 20.0;
v[2] = 3.14;

printf("%lf\n", v[0]); // 읽기 → 1.414

const Vector& cv = v;    // 참조로 const 접근 - 복사 없음
printf("%lf\n", cv[1]); // const 버전 호출 → 20.0
// cv[0] = 99.9;        // ❌ double 반환 - 대입 불가
```

두 버전을 모두 제공해야 `const` 객체에서도 읽기가 가능하다. 범위 검사(`i < size`)를 추가하면 더 안전하다.

operator() — 함수 호출 연산자

operator() 는 **여러 인수**를 받거나 복잡한 타입을 반환하는 것도 가능하다. 수학적 객체를 함수처럼 다룰 수 있어 선형대수 연산에 자주 활용된다.

구현

```
class Vector2D {
    double x, y;
public:
    Vector2D(double _x, double _y) : x(_x), y(_y) {}
    // 인덱스로 성분 접근 (0→x, 1→y)
    double& operator()(int i) { return i == 0 ? x : y; }
    double operator()(int i) const { return i == 0 ? x : y; }
};

class Matrix2D {
    double m[2][2]; // m[행][열]
public:
    Matrix2D(double a, double b,
              double c, double d) {
        m[0][0] = a; m[0][1] = b;
        m[1][0] = c; m[1][1] = d;
    }

    // (i,j) 위치의 원소 반환 / 대입
    double& operator()(int i, int j) { return m[i][j]; }
    double operator()(int i, int j) const { return m[i][j]; }
};
```

사용

```
Vector2D v(3.0, 4.0);
printf("%.0f\n", v(0)); // 3 (x 성분)
printf("%.0f\n", v(1)); // 4 (y 성분)

Matrix2D S(2, 0, 0, 3); // [ 2  0 ]
                        // [ 0  3 ]
printf("%.0f\n", S(0, 0)); // 2 (읽기 - const 버전)
S(0, 0) = 1.0;             // 대입 - non-const 버전 호출
printf("%.0f\n", S(0, 0)); // 1 (변경 확인)
```

operator() 는 인수 시그니처를 다르게 해 **오버로딩**할 수 있다.

Matrix2D 는 (int,int) 로 원소 접근

operator() — 1D 배열로 2D 행렬 구현

double m[4] 로 저장해서 행 우선(row-major)이나 열 우선(column-major) 순서로 메모리에 연속 배치할 수도 있다.

행 우선(row-major) 구현

```
class Matrix2D {
    double m[4]; // [0]=a [1]=b [2]=c [3]=d
                // row-major: m[i*2 + j]

public:
    Matrix2D(double a, double b, double c, double d) {
        m[0] = a; m[1] = b; m[2] = c; m[3] = d;
    }

    // (i,j) → i*2 + j 로 1D 인덱스 변환
    double& operator()(int i, int j) {
        return m[i * 2 + j];
    }

    double operator()(int i, int j) const {
        return m[i * 2 + j];
    }
};
```

메모리 배치

인덱스: [0] [1] [2] [3]
값: a b c d

행렬로 보면:

(0,0)=a (0,1)=b
(1,0)=c (1,1)=d

(i, j) → m[i*2 + j]

열 우선(column-major) 구현

```
class Matrix2D {
    double m[4]; // [0]=a [1]=c [2]=b [3]=d
                // column-major: m[i + j*2]

public:
    Matrix2D(double a, double b, double c, double d) {
        m[0] = a; m[1] = c; m[2] = b; m[3] = d;
    }

    // (i,j) → i + j*2 로 1D 인덱스 변환
    double& operator()(int i, int j) {
        return m[i + j * 2];
    }

    double operator()(int i, int j) const {
        return m[i + j * 2];
    }
};
```

메모리 배치

인덱스: [0] [1] [2] [3]
값: a c b d

행렬로 보면:

(0,0)=a (0,1)=b
(1,0)=c (1,1)=d

(i, j) → m[i + j*2]

클래스 파일 분리

클래스는 보통 **헤더 파일**(.h) 에 선언, **소스 파일**(.cpp) 에 정의를 분리한다.

student.h — 선언

```
// student.h
#pragma once
#include <cstring>

class Student {
public:
    int age;
    char name[32];

    Student(int age, const char* name);
    void greet() const;
};
```

student.cpp — 정의

```
// student.cpp
#include "student.h"
#include <cstdio>

Student::Student(int age, const char* name) {
    this->age = age;
    strcpy(this->name, name);
}

void Student::greet() const {
    printf("안녕하세요, %s입니다.\n", name);
}
```

main.cpp — 사용

```
// main.cpp
#include "student.h"

int main() {
    Student s(20, "홍길동");
    s.greet();
}
```

파일

내용

.h

클래스 선언, 인라인 함수

.cpp

멤버 함수 정의 (클래스명::함수명)

#pragma once

중복 #include 방지

헤더에 구현을 넣으면 여러 .cpp 에서 include할 때 **중복 정의 오류**가 발생한다. 예외: 짧은 inline 함수, 템플릿 함수.

요약

개념	키워드 / 문법	핵심
함수 오버로딩	동일 이름, 다른 매개변수	시그니처(이름+매개변수)로 함수 구별; 반환형 제외
생성자 오버로딩	<code>T()</code> , <code>T(int)</code> , <code>T(const T&)</code>	매개변수 구성이 다른 여러 생성자 정의
복사 생성자	<code>T(const T&)</code>	깊은 복사; 동적 메모리 있으면 직접 구현
복사 대입 연산자	<code>T& operator=(const T&)</code>	자기 대입 방지 + 기존 자원 해제 후 복사
static 멤버	<code>static</code>	클래스에 속함, 모든 객체가 공유
const 멤버 함수	<code>T f() const</code>	객체 상태 변경 불가 계약
연산자 오버로딩	<code>operator+</code> 등	직관적일 때만 사용
friend	<code>friend</code>	private 접근 허용; 최소화